Principios y Mejores Prácticas para Tests Unitarios

Esta guía te ayudará a identificar qué aspectos de tu código merecen tests unitarios y cuáles no, optimizando tu tiempo y esfuerzo en testing.

¿Qué SÍ probar?

1. **Comportamiento público de la función/método.**
   Se refiere a la interfaz externa de tu código, es decir, lo que otros desarrolladores verán y usarán
   Entradas válidas: Que devuelva los resultados esperados para los casos normales.
   Casos límite: Valores extremos, arrays vacíos, strings vacíos.
   Validaciones: Que rechace entradas inválidas correctamente.

2. **Flujos de control importantes.**
   Las diferentes rutas que puede tomar tu código según las condiciones.
   Condicionales: Todos los if/else significativos.
   Bucles: Comportamiento con 0, 1 y múltiples elementos.
   Switch/case: Cada rama importante que produce diferentes resultados.

3. **Manejo de errores.**
   Excepciones esperadas: Que lance errores cuando debe.
   Validación de parámetros: Entrada null, undefined, tipos incorrectos.
   Estados de error: Fallos de red, timeouts, etc.

4. **Estados y transiciones.**
   Para objetos/clases que mantienen estado interno.
   Estado inicial: Valores por defecto.
   Cambios de estado: Transiciones válidas.
   Estado final: Resultado esperado tras operaciones.

¿Qué NO probar?

1. **Detalles de implementación.**
   El "cómo" interno de tu código, que puede cambiar sin afectar el comportamiento externo.
   Métodos privados: Solo si es absolutamente necesario.
   Variables internas no expuestas: No expuestas al usuario de la clase/función.
   Orden específico de llamadas internas: Importa solo que el resultado sea correcto.

2. **Dependencias externas.**
   Servicios, APIs o recursos que están fuera de tu control.
   APIs de terceros: Usar mocks/stubs en lugar de llamadas reales.
   Base de datos: Es testing de integración, no unitario.
   Sistema de archivos: Usar mocks para simular lectura/escritura.

3. **Funcionalidad del framework.**
   Código que ya ha sido probado por sus creadores.
   Métodos nativos de JavaScript: `Array.map()`, `String.split()`, etc.
   APIs del navegador: ejemplo: `localStorage`, `fetch, es mejor usar mocks.
   Librerías de terceros: estas ya fueron probadas, y se asume que funcionan.

Cobertura de casos

Los diferentes escenarios que debes cubrir para tener confianza en tu código.
Happy path: Caso normal exitoso - el flujo principal que esperamos.
Edge cases: Casos límite que pueden romper la lógica.
Error cases: Casos de fallo esperados.
Boundary conditions: Límites exactos de rangos válidos.

Metodología práctica

Existen 2 enfoques complementarios para estructurar tests, que pueden usarse de manera individual o híbrida, cada uno con sus fortalezas: **Técnica AAA** y **Patrón Given-When-Then.**

Técnica AAA (Arrange-Act-Assert)
Este patrón sirve para organizar el código del test de manera estructural brindando una presentación técnica de forma clara y mantenible. Se recomienda para tests simples, funciones puras y casos directos.

```
describe('UserService.createUser', () => {
 it('should create user successfully', () => {
   // ARRANGE: Preparar datos y dependencias
   const userData = { name: 'Ana', email: 'ana@test.com' };
   const mockRepository = jest.fn().mockResolvedValue({ id: 1, ...userData });

   // ACT: Ejecutar la función
   const result = userService.createUser(userData);

   // ASSERT: Verificar resultado
   expect(result).toEqual({ id: 1, name: 'Ana', email: 'ana@test.com' });
 });
});
```

Patrón Given-When-Then (BDD Style)
Este patrón narrativo describe el comportamiento esperado en lenguaje natural, viene del BDD (Behavior-Driven Development o Desarrollo guiado por comportamiento), donde, una aplicación se documenta y diseña en función del comportamiento que los usuarios esperan

experimentar al interactuar con ella. Se recomienda para tests de comportamiento complejo, escenarios con múltiples precondiciones o resultados.
Puede usarse tanto en test unitarios como en tests de integración y/o E2E

```
describe('User Registration', () => {
 it('should successfully register a new user with valid information', () => {
   // GIVEN: Un usuario con información válida
   const validUserData = { name: 'Ana', email: 'ana@test.com' };
   // AND: El sistema está disponible
   mockUserRepository.isAvailable.mockReturnValue(true);

   // WHEN: El usuario intenta registrarse
   const result = userService.registerUser(validUserData);

   // THEN: El usuario debe ser creado exitosamente
   expect(result.success).toBe(true);
   // AND: Debe recibir un ID único
   expect(result.user.id).toBeDefined();
   // AND: Debe recibir un email de confirmación
   expect(emailService.sendWelcomeEmail).toHaveBeenCalledWith(validUserData.email);
 });
});
```

Se pueden combinar brindando la parte técnica y estructural del AAA y la orientación al comportamiento del Given-When-Then.

```
describe('Payment Processing', () => {
 it('should process payment for premium user with sufficient balance', () => {
   // GIVEN: Usuario premium con saldo de 500€
   const user = createPremiumUser({ balance: 500 });
   const payment = { amount: 100, currency: 'EUR' };

   // WHEN: Se procesa el pago
   const result = paymentService.processPayment(user, payment);

   // THEN: El pago debe ser exitoso
   expect(result.status).toBe('success');
   expect(result.newBalance).toBe(400);
   expect(result.fees).toBe(0); // Sin fees para premium
 });
});
```